Department of Computer Science
Faculty of Sciences and Techniques Tangier
Abdelmalek Essaâdi University, Morocco
March 26, 2026

Comparative Study of Deep Learning Video Models for Automatic Criminal Behavior Detection: A Benchmark of Seven Neural Architectures

Supervised by: Pr. Mostapha Ezzyani
Made By: Mohamed EL Attar , Kaouthar Hamdan

•• **ABSTRACT** Automated crime detection from surveillance video is a critical challenge in public safety. In this paper, we present a comprehensive benchmark of seven state-of-the-art deep learning video architectures—I3D, MViT-v2, R(2+1)D, SlowFast, Swin3D, TimeSformer, and X3D—applied to classifying video clips into three criminal behaviour categories: *violence*, *theft*, and *normal*. All models are fine-tuned on a unified multi-source dataset combining RWF-2000, UCF-Crime, and CAMNUVEON (2,974 videos) pre-trained on Kinetics-400 and evaluated on a held-out test set of 492 clips. MViT-v2-S achieves the best performance (90.04% accuracy, macro F1 = 0.8984), followed by I3D (87.22%) and R(2+1)D (85.77%). Models lacking pretrained weights drop 18–38 percentage points, confirming the critical role of Kinetics-400 transfer learning. Our results provide actionable insights for intelligent surveillance system design.

•• **KEY-WORDS** Crime detection, video classification, deep learning, spatio-temporal models, 3D convolutions, video transformers, benchmark, violence detection, theft detection, MViT-v2, I3D, SlowFast, Swin3D.

I. INTRODUCTION

In the digital age, the proliferation of closed-circuit television (CCTV) infrastructure has generated surveillance footage that exceeds human capacity to monitor in real time. Manual review is expensive and prone to fatigue, resulting in missed criminal events. The automation of crime detection has thus emerged as a high-priority challenge at the intersection of computer vision, deep learning, and public safety.

Traditional approaches relied on hand-crafted features—optical flow, HOG, and background subtraction—combined with SVMs [16]. While effective in controlled settings, these methods lack the capacity to generalise across diverse real-world conditions. The advent of 3D convolutional networks (I3D [1], R(2+1)D [2]) and video transformers (TimeSformer [7], Swin3D [5], MViT-v2 [6]) brought major advances, yet comparative benchmarks on crime-specific datasets remain scarce.

This work presents a unified benchmark of seven architectures on three merged datasets, with three main contributions:

1. A unified dataset (2,974 videos) from RWF-2000, UCF-Crime, and CAMNUVEON mapped to three classes: *violence*, *theft*, *normal*.
2. A consistent fine-tuning and evaluation pipeline across all seven models.
3. Comprehensive comparison of macro metrics, per-class breakdowns, and training dynamics to guide intelligent surveillance deployment.

II. RELATED WORK

A. Traditional Methods

Background modelling (GMM) [16], optical flow, and trajectory analysis enabled early violence detection but required extensive domain engineering and were sensitive to occlusion and camera motion.

B. Machine Learning Approaches

SVMs with Dense Trajectories [15] and Fisher Vectors achieved competitive results on benchmarks. Random forests and boosting were applied to crowd violence detection. However, these methods struggled with the semantic complexity of criminal behaviour.

C. Deep Learning for Video

The two-stream framework [8] established the value of modelling appearance and motion in parallel. C3D [9] and I3D [1] enabled spatio-temporal feature learning from raw clips. R(2+1)D [2] factorised 3D convolutions for efficiency. SlowFast [3] introduced dual-pathway processing at two temporal resolutions. X3D [4] proposed multi-axis expansion for compact video models. On the transformer side, TimeSformer [7], Swin3D [5], and MViT-v2 [6] brought global temporal attention to video understanding.

D. Crime and Violence Detection

UCF-Crime [10] introduced weakly supervised anomaly detection across 13 crime categories. RWF-2000 [11] provided a large-scale fight detection dataset from surveillance cameras. Prior work used C3D and two-stream networks; direct comparisons of modern transformer vs. CNN architectures on a unified three-class task remain understudied — a gap this work addresses.

III. METHODOLOGY

A. Dataset Construction

Three publicly available datasets are merged into a unified corpus of **2,974 videos** (Table 1). RWF-2000 [11] provides binary fight/non-fight surveillance clips; UCF-Crime [10] covers 13 anomaly categories; and CamNuvem [12] is a Brazilian CCTV robbery dataset.

Table 1. Source datasets and class mapping.

Dataset	Domain	Original Label	Mapped
RWF-2000	Violence	Fight	violence
		NonFight	normal
UCF-Crime	Violence/Theft	Shoplifting, Stealing, Robbery, Burglary	theft
		Normal	normal
CAMNUVEON	Theft/Normal	Anomaly	theft
		Normal	normal

B. Preprocessing Pipeline

A common preprocessing pipeline is applied uniformly to all models and datasets across the five following stages.

(1) Frame extraction. A fixed temporal window of T frames is sampled from each video clip. For most models, frames are drawn uniformly over the full video duration. For UCF-Crime clips—which contain long neutral segments before and after the criminal event—all models except I3D apply a centre-biased strategy that restricts sampling to the 25%–75% temporal interval, reducing the proportion of uninformative background frames in the input window. I3D instead uses full-video uniform sampling to maximise temporal context.

(2) Spatial resizing. Each frame is resized to the target spatial resolution of the model. The majority of architectures operate at 224×224 pixels, which corresponds to the standard Kinetics-400 pretraining resolution. R(2+1)D is the sole exception, using a reduced resolution of 112×112 to accommodate its larger batch size of 8 within GPU memory constraints.

(3) Normalisation. Pixel values are first scaled to the range [0, 1], then channel-wise standardised using the per-channel mean and standard deviation of the Kinetics-400 training set. This ensures that fine-tuned models receive inputs that are statistically aligned with the distribution seen during pretraining, accelerating convergence and stabilising training.

(4) Tensor formatting. For single-input models, the output tensor follows the shape (C, T, H, W). SlowFast requires two input tensors produced from the same clip: a *Slow* tensor of 8 frames and a *Fast* tensor of 32 frames ($\alpha = 4$), handled via a custom `slowfast_collate` function.

(5) Dataset splitting. Stratified sampling with `random_state=42` partitions the data into 80% training, 20% validation, and a held-out test set comprising 15% of the total corpus. For X3D, a $K = 2$ StratifiedKFold is applied on the combined 85% train–validation pool. All DataLoaders use `pin_memory=True`; `drop_last=True` is set for training loaders to ensure consistent batch sizes throughout training.

C. Models

Seven Kinetics-400 pre-trained architectures are fine-tuned for the three-class task. Table 2 summarises key configurations.

Table 2. Model configurations.

Model	Variant	Type	Fr.	Res.	Opt.
I3D	R3D-18	3D CNN	16	224	Adam
MViT-v2	mvit_v2_s	Transformer	16	224	AdamW
R(2+1)D	r2plus1d_18	Factored CNN	16	112	SGD
SlowFast†	custom R18	Dual-Path	8/32	224	Adam
Swin3D	swin3d_t	Transformer	16	224	Adam
TimeSformer	tsf-base-k400	ViT	8	224	AdamW
X3D†	custom 3D CNN	Expanding	16	224	Adam

† Custom fallback — no pretrained weights.

1. I3D — Inflated 3D ConvNet

I3D [1] inflates 2D filters into 3D. We use `r3d_18` (33.2M parameters) with uniform full-video sampling (Adam, StepLR, lr=1e-4, 10 epochs).

2. MViT-v2 — Multiscale Vision Transformer

MViT-v2 [6] applies hierarchical pooling attention (≈36M params). AdamW with CosineAnnealingLR is standard for transformers (lr=1e-4, wd=1e-4).

3. R(2+1)D — Factored Spatiotemporal CNN

R(2+1)D [2] decomposes 3D convolutions into 2D spatial + 1D temporal (≈14M params). Resolution 112×112, batch size 8, SGD (momentum=0.9).

4. SlowFast — Dual-Pathway Network

SlowFast [3]: Slow pathway (8 frames) + Fast pathway (32 frames, α=4). *Note*: original pretrained SlowFast-R50 was unavailable on Kaggle Python 3.12 (`fvcore`/`pytorchvideo` missing); a custom R18 fallback without pretrained weights was used. Results are not representative of the original architecture.

5. Swin3D — 3D Shifted Window Transformer

Swin3D [5] applies shifted-window attention in 3D (≈28M params). The Tiny variant (`swin3d_t`) serves as the **baseline** of this benchmark.

6. TimeSformer — Divided Space-Time Attention

TimeSformer [7] separates spatial and temporal attention (121.3M params). Loaded from HuggingFace with `num_labels=3`. Batch size 2 due to memory.

7. X3D — Expanding Architecture

X3D [4] expands a 2D baseline along five axes. Original `x3d_m` unavailable; custom fallback used with $K = 2$ StratifiedKFold, 5 epochs.

D. Training Protocol

All models: cross-entropy loss; early stopping patience = 3; Kaggle NVIDIA P100/T4.

IV. RESULTS & IMPLEMENTATION

A. Overall Performance

Table 3 ranks all seven models by test accuracy. MViT-v2-S achieves the best performance (90.04%, macro F1 = 0.8984). Among pretrained models, the top four all exceed 84%, while the fallback models collapse to 71.75% and 52.03%, confirming the critical role of Kinetics-400 pretraining.

Table 3. Main benchmark results. Best values **bolded**. † = custom fallback (no pretraining).

Rk	Model	Acc %	F1	Prec	Rec
1	**MViT-v2-S**	**90.04**	**.8984**	**.8963**	**.9009**
2	I3D (R3D-18)	87.22	.8830	.8744	.8944
3	R(2+1)D-18	85.77	.8550	.8527	.8581
4	Swin3D-T	84.76	.8458	.8442	.8498
5	TimeSformer-B	77.03	.7670	.7670	.7696
6	SlowFast†	71.75	.7213	.7155	.7315
7	X3D†	52.03	.4837	.4985	.4829

Test set: 492 samples (I3D: 360). * = early-stopped.

B. Per-Class F1-Score Analysis

Table 4 breaks down F1-scores per class. *Theft* is consistently the hardest class across all models. I3D achieves a remarkable theft F1 of 0.9291 (recall 98.3%), benefiting from full-video uniform sampling that captures prolonged subtle motions. MViT-v2 leads on violence (F1 = 0.9346) thanks to its multi-scale attention.

Table 4. Per-class F1-score breakdown.

Model	Viol.	Theft	Norm.	Macro
MViT-v2-S	**.9346**	.8617	**.8988**	**.8984**
I3D (R3D-18)	.8618	**.9291**	.8581	.8830
R(2+1)D-18	.8997	.8095	.8558	.8550
Swin3D-T	.8517	.8320	.8537	.8458
TimeSformer-B	.7900	.7347	.7762	.7670
SlowFast†	.7184	.7509	.6946	.7213
X3D†	.4681	.3628	.6201	.4837

C. Per-Class Precision & Recall

Table 5 reports per-class precision and recall. I3D achieves the highest single-class score in the benchmark: theft recall = 0.9833 (59/60 correct). MViT-v2 leads on violence recall (0.9533) and normal precision (0.9139). The theft class shows the widest precision-recall gap across models, reflecting semantic ambiguity between shoplifting and normal shopping. Figure 1 visualises the full confusion matrices for all seven models.

Table 5. Per-class precision (P) and recall (R). VIO = Violence, THE = Theft, NOR = Normal.

Model	V-P	V-R	T-P	T-R	N-P	N-R
MViT-v2-S	.917	**.953**	.858	.865	**.914**	.884
I3D	.851	.873	**.881**	**.983**	.892	.827
R(2+1)D	.874	.927	.810	.810	.874	.838
Swin3D	.808	.900	.839	.825	.886	.824
TimeSformer	.746	.840	.756	.714	.799	.755
SlowFast†	.698	.740	.706	.802	.742	.653
X3D†	.500	.440	.438	.310	.557	.699

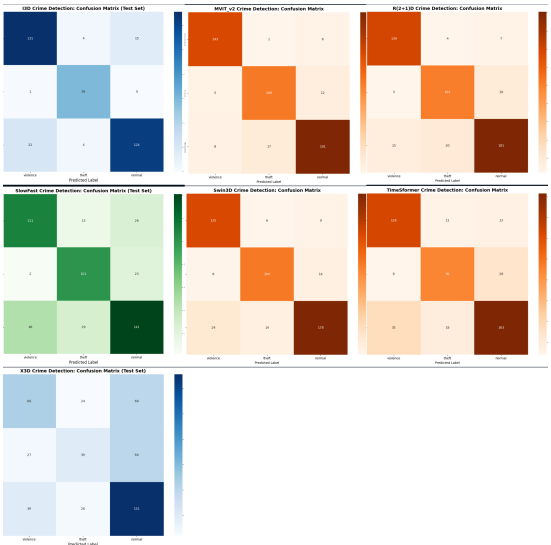


Figure 1. Confusion matrices for all seven models on the held-out test set (rows = true labels, columns = predicted labels; classes: violence, theft, normal). MViT-v2 and I3D show the most concentrated diagonals. Fallback models (SlowFast†, X3D†) display the largest off-diagonal mass.

D. Training Dynamics

MViT-v2 converged rapidly, reaching its best val accuracy of 91.02% at epoch 3 before early stopping at epoch 6. I3D followed a steady trajectory, peaking at 91.67% before stopping at epoch 7. R(2+1)D peaked at epoch 4 (90.48%) with a smooth decreasing loss under SGD. Swin3D exhibited a non-monotone validation curve (77.9% at epoch 2, recovering to 88.9% at epoch 7), suggesting sensitivity to StepLR on transformer architectures. TimeSformer showed the most volatile training (oscillating 60%–79.7%), attributed to its 121.3M parameters, batch size 2, and data scarcity. Fallback models showed slow convergence confirming that training from scratch on <3,000 videos is insufficient. Figure 2 illustrates the loss and accuracy curves for all models across training epochs.

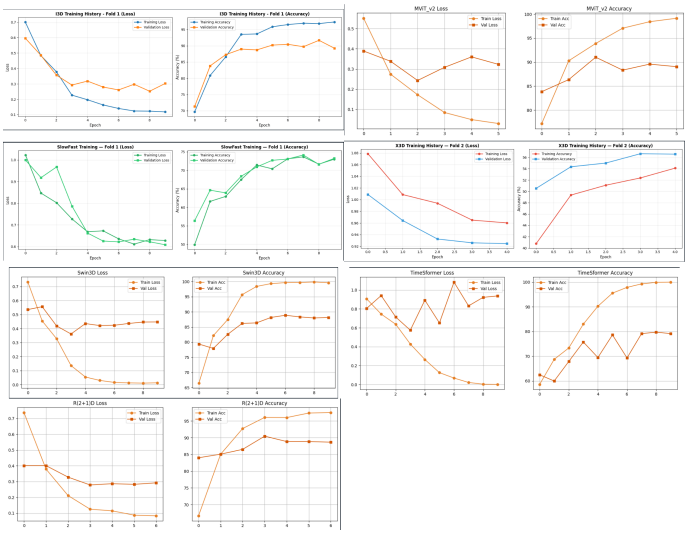


Figure 2. Training and validation loss/accuracy curves for all seven models. Top row: I3D (blue) and MViT-v2 (orange). Middle row: SlowFast† (green) and X3D† (red/blue). Bottom rows: Swin3D, TimeSformer, and R(2+1)D. Note the volatile TimeSformer curves vs. the smooth R(2+1)D convergence under SGD.

V. DISCUSSION

A. CNN vs. Transformer Architectures

The performance gap between CNN and transformer models is modest for models with Kinetics-400 pretraining. I3D (33.2M, 87.22%) and R(2+1)D (14M, 85.77%) both outperform the transformer baseline Swin3D-T (28M, 84.76%), showing that strong convolutional inductive biases remain competitive on small crime detection datasets. MViT-v2 breaks this trend: its multiscale pooling design provides a +5.28% accuracy gain over Swin3D-T (both transformers), highlighting the value of hierarchical spatio-temporal abstraction.

B. Critical Role of Pretraining

The most striking result is the performance collapse of fallback models: SlowFast (71.75%) and X3D (52.03%) vs. 84–90% for pretrained models. This **18–38 percentage point gap** confirms that Kinetics-400 pretraining provides essential features that cannot be learned from 2,974 videos alone.

C. Class-Level Difficulty

Across all pretrained models, *theft* is the hardest class (lowest F1 for all but I3D). Theft and normal shopping are visually near-identical in gross motor actions; the distinguishing cues are temporally sparse within 16-frame clips. *Violence*, by contrast, has salient motion signatures (rapid limb movement, contact) yielding high recall ($\geq$0.87) for the top four models. The *normal* class achieves high precision but lower recall for some models (e.g., I3D: P=0.892, R=0.827), indicating false negatives where normal clips are misclassified as criminal.

D. Parameter Efficiency

R(2+1)D (14M params, 85.77%) is the most parameter-efficient model. TimeSformer (121.3M params, 77.03%) underperforms despite its large capacity, attributed to its 8-frame input, batch size 2, and overfitting on the small dataset. For resource-constrained deployments, R(2+1)D offers the best efficiency–accuracy trade-off.

E. Limitations

- **Library incompatibilities:** SlowFast and X3D results use custom fallbacks without pretrained weights; they do not reflect original architecture performance.
- **Dataset size:** ≈3,000 videos is small; larger datasets would benefit all models, especially large transformers.
- **Class imbalance:** Test set: 216 normal vs. 126 theft samples.
- **Evaluation heterogeneity:** I3D evaluated on 360 vs. 492 samples for other models introduces a minor inconsistency.
- **Epoch budget:** 10 epochs may be insufficient for TimeSformer-class models.

VI. CONCLUSION

We presented a systematic benchmark of seven deep learning video architectures for automated crime detection on a unified multi-source dataset (RWF-2000, UCF-Crime, CAMNUVEON). Under consistent fine-tuning and evaluation, **MViT-v2-S achieves the best overall performance** (90.04%, macro F1 = 0.8984), followed by I3D (87.22%) and R(2+1)D (85.77%). CNN-based architectures with Kinetics-400 pretraining remain strong competitors to transformers at this data scale, with R(2+1)D offering the best parameter efficiency (14M params, 85.77%). The most consequential finding is the **critical role of pretraining**: models without Kinetics-400 weights drop 18–38 percentage points, confirming that transfer learning from large-scale action recognition corpora is a prerequisite for crime detection performance.

Per-class analysis reveals that *theft* is the hardest class due to visual similarity with normal behaviour, while *violence* is reliably detected via distinctive motion signatures. TimeSformer underperforms relative to its parameter count, underscoring that model scale must match data availability.

Future directions include: temporal action localisation within long clips; larger multi-source datasets for large transformer training; model compression for real-time edge deployment; and semi-supervised learning to leverage unlabelled surveillance footage.

REFERENCES

[1] J. Carreira and A. Zisserman, "Quo vadis, action recognition? A new model and the kinetics dataset," in *Proc. IEEE/CVF CVPR*, 2017, pp. 6299–6308.

[2] D. Tran *et al.*, "A closer look at spatiotemporal convolutions for action recognition," in *Proc. IEEE/CVF CVPR*, 2018, pp. 6450–6459.

[3] C. Feichtenhofer, H. Fan, J. Malik, and K. He, "SlowFast networks for video recognition," in *Proc. IEEE/CVF ICCV*, 2019, pp. 6202–6211.

[4] C. Feichtenhofer, "X3D: Expanding architectures for efficient video recognition," in *Proc. IEEE/CVF CVPR*, 2020, pp. 203–213.

[5] Z. Liu *et al.*, "Video swin transformer," in *Proc. IEEE/CVF CVPR*, 2022, pp. 3202–3211.

[6] Y. Li *et al.*, "MViTv2: Improved multiscale vision transformers," in *Proc. IEEE/CVF CVPR*, 2022, pp. 4804–4814.

[7] G. Bertasius, H. Wang, and L. Torresani, "Is space-time attention all you need for video understanding?" in *Proc. ICML*, 2021.

[8] K. Simonyan and A. Zisserman, "Two-stream convolutional networks for action recognition in videos," in *Advances in NeurIPS*, 2014, pp. 568–576.

[9] D. Tran *et al.*, "Learning spatiotemporal features with 3D convolutional networks," in *Proc. IEEE ICCV*, 2015, pp. 4489–4497.

[10] W. Sultani, C. Chen, and M. Shah, "Real-world anomaly detection in surveillance videos," in *Proc. IEEE/CVF CVPR*, 2018, pp. 6479–6488.

[11] M. Cheng, K. Cai, and M. Li, "RWF-2000: An open large scale video database for violence detection," in

Proc. IEEE ICPR, 2021, pp. 4183–4190.

[12] D. D. de Paula, D. H. P. Salvadeo, and D. M. N. de Araujo, "CamNuvem: A robbery dataset for video anomaly detection," *Sensors*, vol. 22, no. 24, art. 10016, 2022, doi: 10.3390/s222410016.

[13] K. Soomro, A. R. Zamir, and M. Shah, "UCF101: A dataset of 101 human actions classes from videos in the wild," *arXiv:1212.0402*, 2012.

[14] W. Kay *et al.*, "The kinetics human action video dataset," *arXiv:1705.06950*, 2017.

[15] H. Wang and C. Schmid, "Action recognition with improved trajectories," in *Proc. IEEE ICCV*, 2013, pp. 3551–3558.

[16] C. Stauffer and W. E. L. Grimson, "Adaptive background mixture models for real-time tracking," in *Proc. IEEE CVPR*, vol. 2, 1999, pp. 246–252.

[17] Z. Liu *et al.*, "Swin Transformer: Hierarchical vision transformer using shifted windows," in *Proc. IEEE/CVF ICCV*, 2021, pp. 10012–10022.

ABDELMALEK ESSAÂDI UNIVERSITY

Faculty of Sciences and Techniques of Tangier

Department of Computer Science

TECHNICAL COMPANION REPORT

Neural Networks

Technical Implementation of Deep Learning Video Models for Automatic Criminal Behavior Detection

From raw surveillance video to a deployable prediction service

Architecture, training workflow, evaluation and deployment

7	**2,974**	**3**
video architectures	curated videos	behavior classes

Authors:

Mohamed EL Attar

Kaouthar Hamdan

Supervisor:

Prof. Mostapha Ezzyani

Master in Artificial Intelligence & Data Science

Academic Year: 2025–2026

Contents

1 Purpose and Scope **1**

1.1 Project Artifacts 1

2 Technology Stack **1**

3 Software Architecture **2**

3.1 Component Responsibilities 2

4 Dataset Construction Workflow **2**

5 End-to-End Video Pipeline **3**

5.1 Temporal Sampling 4

5.2 Spatial and Tensor Processing 4

6 Model Architecture Layer **4**

6.1 Model Adaptation Workflow 5

6.2 SlowFast Dual-Path Architecture 5

7 Training and Evaluation Architecture **5**

7.1 Evaluation Flow 6

8 Experimental Results **6**

9 Inference and Deployment Architecture **7**

9.1 Deployment Components 7

10 Reproducibility, Limitations and Improvements **8**

10.1 Reproducibility Controls 8

10.2 Current Limitations 8

10.3 Recommended Evolution 8

11 Conclusion **8**

1. Purpose and Scope

The research paper *Comparative Study of Deep Learning Video Models for Automatic Criminal Behavior Detection* answers a scientific question: which video architecture performs best on a unified crime-detection benchmark? This report answers the practical question behind it: how was the complete system organized and how can it move from an experiment to a usable service?

The starting point is a surveillance video; the expected output is a clear probability distribution over **normal activity**, **theft** and **violence**. To make the comparison meaningful, the seven architectures share one processing logic wherever possible. Model-specific constraints—such as the number of frames or image resolution—remain explicit rather than being hidden inside the notebooks.

Research result used as the technical reference

The unified corpus contains **2,974 videos**. MViT-v2-S achieves the best overall performance with **90.04% accuracy** and a **0.8984 macro F1-score**. I3D reaches 87.22%, while R(2+1)D-18 reaches 85.77%. The architecture presented in this report explains the processing chain that produced these results.

1.1 Project Artifacts

Artifact	Technical purpose
`Codes/*.ipynb`	One experimental workflow for each video model.
`Criminal Behavior Detection _Papier.pdf`	Scientific methodology and benchmark results.
`Datasets_info.pdf`	Dataset descriptions and bibliographic references.
`requirements.txt`	Python and deep learning dependencies.
`DEPLOYMENT.md`	Production, API, Docker and GPU recommendations.

Table 1: Main project artifacts.

2. Technology Stack

The stack was chosen to keep experimentation flexible without losing sight of deployment. Python and PyTorch form the core. OpenCV and FFmpeg take care of the less glamorous but essential work of decoding videos reliably. TorchVision, PyTorchVideo and Hugging Face provide the model families, while scikit-learn keeps the data splits and evaluation protocol consistent.

Layer	Technologies	Responsibility
Data management	Python, Pandas, NumPy	File discovery, source merging, labels and metadata.
Video processing	OpenCV, FFmpeg	Decoding, frame sampling, resizing and format validation.
Deep learning	PyTorch, TorchVision, PyTorchVideo	Datasets, loaders, transfer learning and GPU execution.
Transformers	Hugging Face Transformers	TimeSformer loading and adaptation.
Evaluation	scikit-learn	Stratified splits, accuracy, precision, recall and F1-score.
Deployment	FastAPI, Uvicorn, Docker, CUDA	Model serving and GPU-enabled inference.
Persistence	PostgreSQL, optional	Request, prediction and latency logging.

Table 2: Technology stack from experimentation to deployment.

Execution environment. The experiments target an NVIDIA GPU, notably through Kaggle environments. GPU memory directly determines the spatial resolution, number of frames and batch size supported by each architecture.

3. Software Architecture

The project currently lives in seven notebooks, one for each architecture. Behind that experimental format, however, the same responsibilities appear again and again: discovering data, preparing clips, building a model, training it, measuring its behavior and saving the best result. The diagram below makes that shared structure visible. It also provides a natural blueprint for turning the notebooks into maintainable Python modules later on.

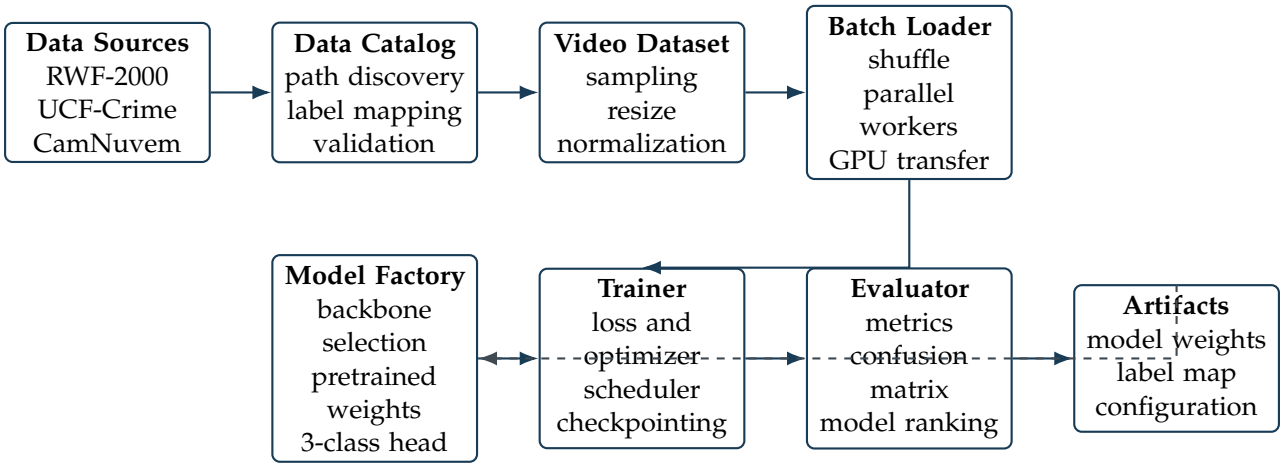


Figure 1: Logical software architecture shared by the seven experiments.

3.1 Component Responsibilities

Component	Input	Output
Data catalog	Dataset root folders	Validated table of video paths and normalized labels.
Video dataset	One path and one label	Fixed-size spatio-temporal tensor and class index.
Batch loader	Dataset and loading policy	Mini-batches ready for CPU–GPU transfer.
Model factory	Model name and class count	Adapted video backbone with a three-class output head.
Trainer	Model, batches and hyperparameters	Best checkpoint and training history.
Evaluator	Checkpoint and held-out test set	Global metrics, class metrics and confusion matrix.
Artifact manager	Best experiment state	Deployable weights, labels and preprocessing configuration.

Table 3: Interfaces between the logical software components.

4. Dataset Construction Workflow

No single source covers the full target problem, so the corpus is assembled from three complementary public datasets. This is more than a file-merging operation: every source uses its own labels and conventions. A dedicated harmonization stage creates one vocabulary that every model can understand.

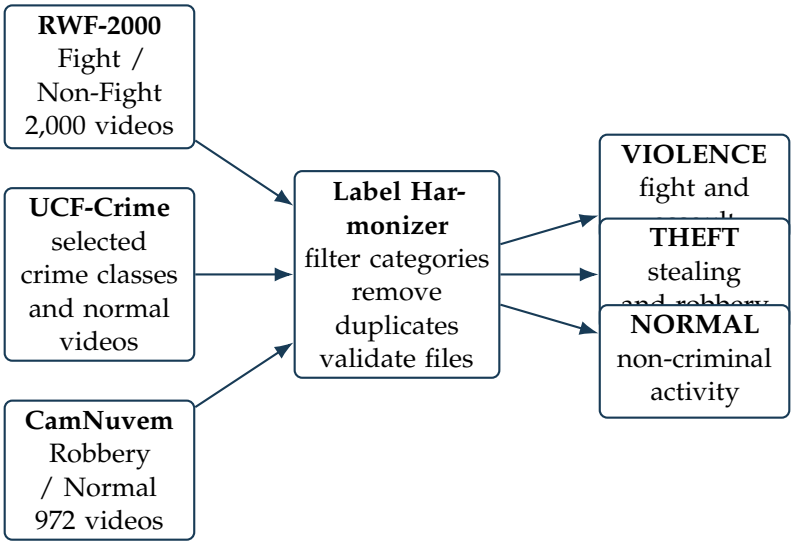


Figure 2: Dataset fusion and label harmonization architecture.

Dataset	Source volume	Selected labels	Unified classes
RWF-2000	2,000	Fight, Non-Fight	violence, normal
UCF-Crime	subset	Shoplifting, Stealing, Robbery, Burglary, Normal	theft, normal
CamNuvem	972	Anomaly, Normal	theft, normal

Table 4: Mapping of source labels to the target taxonomy.

After filtering, **2,974 videos** remain. Before any model sees them, the pipeline checks that each file exists, can be opened, contains frames and has not already been registered. Class balance is reviewed at the same stage. The test set is then isolated through stratified sampling, protecting it from the training process. Most models use 492 test clips; I3D uses 360, so its result should be compared with appropriate caution.

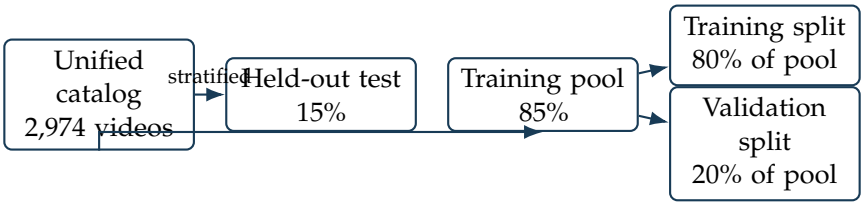


Figure 3: Reproducible train, validation and test partitioning.

5. End-to-End Video Pipeline

Every model follows the same journey from compressed video to GPU-ready tensor. Only the details that genuinely belong to the architecture—temporal length, spatial resolution and input structure—are allowed to change.

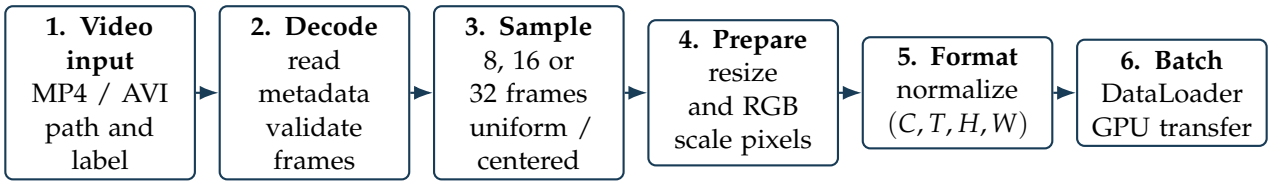


Figure 4: Common video ingestion and preprocessing pipeline.

5.1 Temporal Sampling

Rather than processing every frame, the pipeline selects a compact temporal summary of each video. Most models receive 16 frames. TimeSformer is limited to 8 to control memory consumption, while SlowFast deliberately creates two views: 8 sparse frames for context and 32 dense frames for motion. Long UCF-Crime clips often contain substantial neutral footage, so most experiments sample from the central 25–75% interval. I3D keeps the full timeline to retain broader context.

5.2 Spatial and Tensor Processing

Once selected, frames are converted to RGB, resized and scaled to $[0, 1]$. Normalization follows the distribution expected by the Kinetics-400 pretrained weights; this small detail is essential for stable transfer learning. Finally, the image sequence (T, H, W, C) becomes a PyTorch video tensor (C, T, H, W), and the DataLoader adds the batch dimension B.



Figure 5: Transformation from decoded frames to a model-ready batch.

During experimentation, an unreadable frame is replaced with a neutral tensor so that one damaged video does not interrupt an entire training epoch. A production service must also record the incident; otherwise, a decoding failure could be mistaken for genuinely dark footage.

6. Model Architecture Layer

The benchmark brings together two ways of understanding motion. Video CNNs learn local spatio-temporal patterns through convolution, while video Transformers connect information across patches and time through attention. A shared model layer absorbs their library-specific differences and presents the trainer with the same three-class interface.

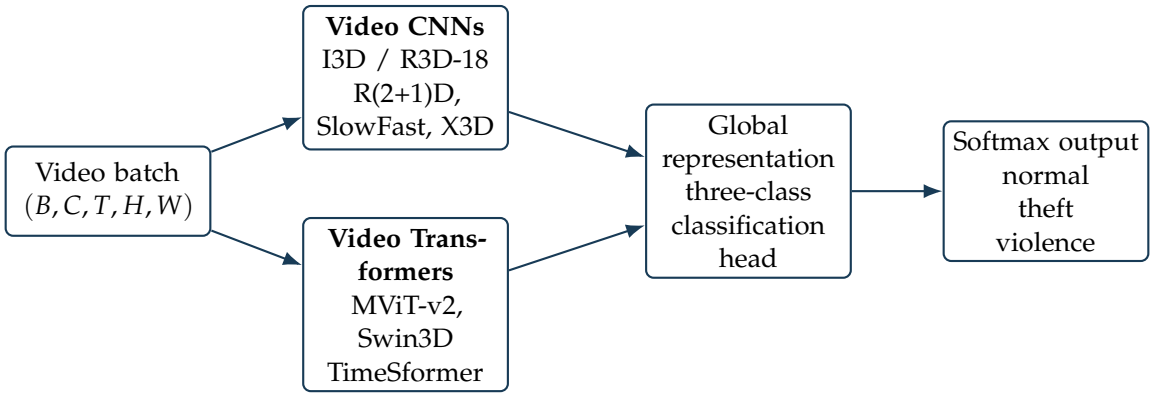


Figure 6: Unified model interface for CNN and Transformer backbones.

Model	Main mechanism	Frames	Resolution	Batch	Optimizer
I3D / R3D-18	Joint 3D spatio-temporal convolution	16	224	4	Adam
MViT-v2-S	Hierarchical multiscale attention	16	224	4	AdamW
R(2+1)D-18	Spatial 2D followed by temporal 1D convolution	16	112	8	SGD
SlowFast*	Slow semantic path and fast motion path	8+32	224	4	Adam
Swin3D-T	Shifted local attention windows in 3D	16	224	4	Adam
TimeSformer-B	Divided spatial and temporal attention	8	224	2	AdamW
X3D*	Progressive expansion of an efficient video network	16	224	4	Adam

Table 5: Model configurations identified in the experimental notebooks. *Fallback implementation without pretrained weights in the final experiment.

6.1 Model Adaptation Workflow

Whenever compatible weights are available, training starts from Kinetics-400 rather than from random initialization. The original 400-action output layer is replaced by a compact head for the three target behaviors. This preserves the motion features learned from a large action-recognition corpus while adapting the final decision boundary to the project data.

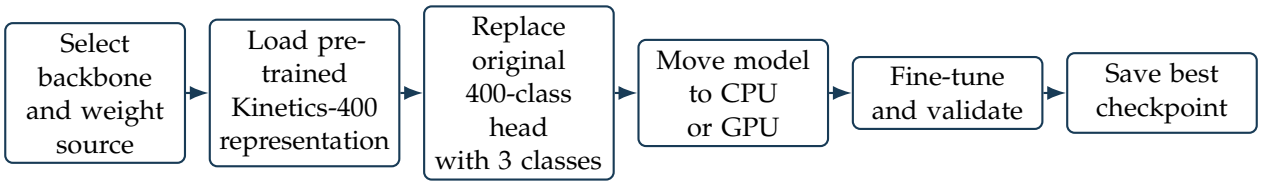


Figure 7: Backbone adaptation and transfer-learning workflow.

6.2 SlowFast Dual-Path Architecture

SlowFast looks at one event at two speeds. Its slow pathway asks what is present in the scene; its fast pathway pays closer attention to how things move. The two feature streams meet just before classification, combining context and motion in a single prediction.

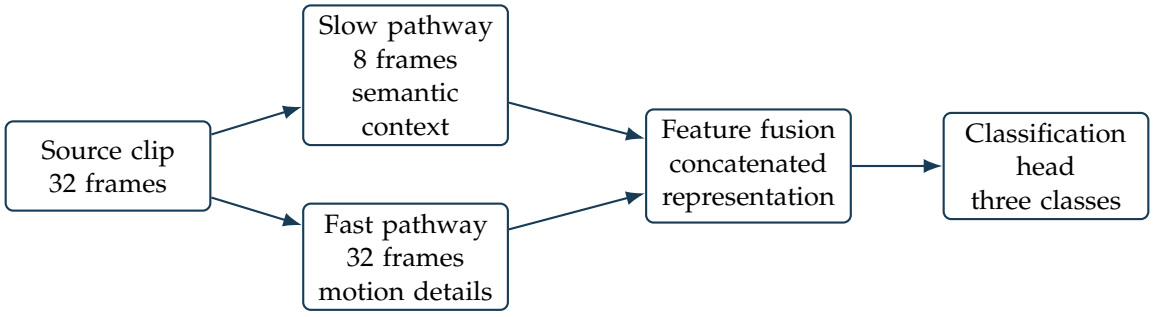


Figure 8: Logical dual-path design of SlowFast.

The original SlowFast-R50 and X3D pretrained implementations were unavailable in the experiment environment because of library compatibility constraints. Fallback networks trained without Kinetics-400 initialization were therefore used. Their scores are useful as implementation diagnostics but are not fully representative of the original pretrained architectures.

7. Training and Evaluation Architecture

Training is treated as a controlled cycle rather than a sequence of isolated cells. The controller receives prepared batches and a configured backbone, updates the model on the training set, checks it on unseen validation data and keeps only the checkpoint that genuinely improves performance.

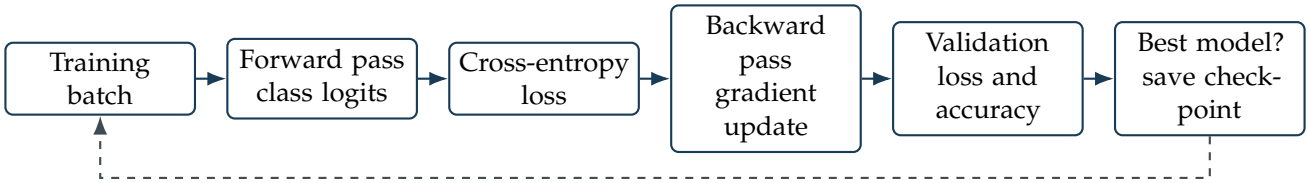


Figure 9: Training epoch and validation control flow.

Model group	Learning rate	Scheduler	Epochs	Main rationale
I3D, Swin3D, SlowFast	10^{-4}	Step decay	10	Stable staged learning-rate reduction.
MViT-v2, TimeSformer	10^{-4}	Cosine decay	10	AdamW and weight decay for Transformers.
R(2+1)D	10^{-3}	Step decay	10	SGD with 0.9 momentum.
X3D*	10^{-4}	Step decay	5	Two-fold validation of the fallback model.

Table 6: Main optimization configurations.

7.1 Evaluation Flow

Only the best validation checkpoint reaches the final test. The evaluator then collects every prediction and looks beyond a single accuracy value: it measures class-level precision, recall and F1-score, builds the confusion matrix and places the model in the final benchmark.

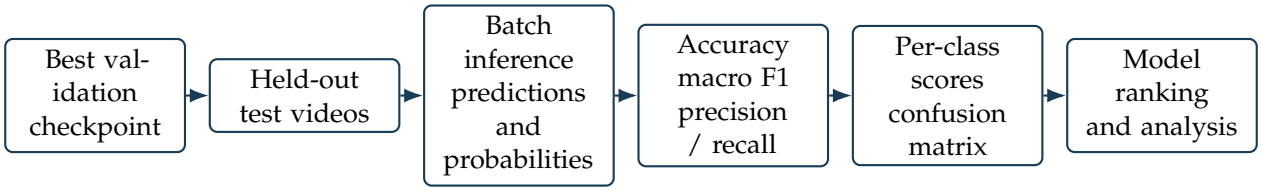


Figure 10: Test evaluation and benchmark generation workflow.

8. Experimental Results

Accuracy offers a useful first impression, but it can hide poor behavior on a smaller or more difficult class. For that reason, the benchmark gives equal attention to macro F1, precision and recall, both globally and per class.

$$\text{Precision} = \frac{TP}{TP + FP}, \qquad \text{Recall} = \frac{TP}{TP + FN}, \qquad F_1 = 2\frac{\text{Precision} \times \text{Recall}}{\text{Precision} + \text{Recall}}.$$

Rank	Model	Accuracy	Macro F1	Precision	Recall
1	MViT-v2-S	**90.04%**	**0.8984**	0.8963	0.9009
2	I3D / R3D-18	87.22%	0.8830	0.8744	0.8944
3	R(2+1)D-18	85.77%	0.8550	0.8527	0.8581
4	Swin3D-T	84.76%	0.8458	0.8442	0.8498
5	TimeSformer-B	77.03%	0.7670	0.7670	0.7696
6	SlowFast*	71.75%	0.7213	0.7155	0.7315
7	X3D*	52.03%	0.4837	0.4985	0.4829

Table 7: Main results reported in the research paper.

Technical interpretation

MViT-v2-S provides the strongest overall result. I3D achieves the best theft recall at 98.33%, correctly identifying 59 of 60 theft samples in its test configuration. R(2+1)D-18 offers the strongest efficiency–accuracy compromise with approximately 14 million parameters. The 18–38 percentage-point decline of the non-pretrained fallback models confirms that Kinetics-400 transfer learning is a critical system component.

Theft remains the most difficult class because shoplifting can resemble normal shopping for most of a clip and may contain only a short discriminative event. Violence usually presents stronger motion patterns and is therefore detected more reliably by the best models.

9. Inference and Deployment Architecture

The production architecture turns the research workflow into a predictable service. A client submits a video or URL; the API validates it, rebuilds exactly the preprocessing used during training, runs the selected checkpoint and returns an understandable result. Monitoring sits beside inference, not after it, so failures and changes in incoming data remain visible.

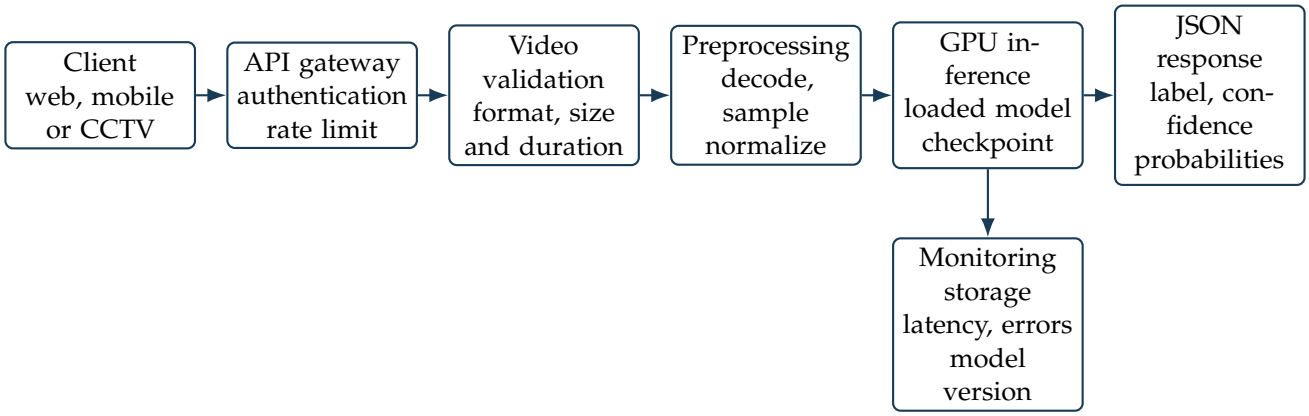


Figure 11: Target inference service architecture.

9.1 Deployment Components

Component	Responsibility
FastAPI / Uvicorn	HTTP interface, request validation and response serialization.
OpenCV / FFmpeg	Reliable video decoding and temporal frame extraction.
PyTorch runtime	Model loading, GPU inference and probability generation.
Docker / CUDA	Reproducible container and access to NVIDIA acceleration.
PostgreSQL, optional	Prediction logs, latency, failures and model version history.
Monitoring	Error rate, processing time, class distribution and data drift.

Table 8: Responsibilities of the deployment layer.

A checkpoint is not deployable on its own. It must travel with its class map and preprocessing configuration; separating them risks silently changing the meaning of a prediction. The service loads this complete package once at startup, performs inference without gradients and returns the label, confidence, full probability vector and model version.

10. Reproducibility, Limitations and Improvements

10.1 Reproducibility Controls

1. fix Python, NumPy and PyTorch random seeds;
2. persist the exact train, validation and test catalogs;
3. freeze Python, PyTorch, TorchVision, CUDA and Transformers versions;
4. store the best checkpoint with its label and preprocessing configuration;
5. evaluate all models on the same test set whenever possible;
6. publish global metrics, per-class metrics and confusion matrices.

10.2 Current Limitations

- SlowFast and X3D use fallback implementations without pretrained weights;
- fewer than 3,000 videos are insufficient for a large model such as TimeSformer;
- I3D does not use exactly the same number of test clips as the other models;
- clip-level classification does not localize the start and end of an event;
- systematic real-time latency and production drift measurements remain future work.

10.3 Recommended Evolution

The next iteration should turn the shared notebook logic into reusable modules that mirror Section 3. Restoring official SlowFast and X3D weights would make their comparison fairer, while a single test protocol would remove the current I3D inconsistency. Mixed precision can reduce training cost. On the deployment side, the project should report frames per second, GPU memory and end-to-end latency. R(2+1)D is the most promising candidate for ONNX/TensorRT export and edge optimization.

11. Conclusion

This project is more than a ranking of neural networks. It is a complete video understanding chain, built from three datasets and carried through sampling, preprocessing, transfer learning, evaluation and deployment design. The work shows that a strong backbone matters, but so do the quieter engineering choices: consistent labels, appropriate temporal sampling, Kinetics-compatible inputs and traceable artifacts.

MViT-v2-S delivers the strongest overall result. I3D stands out on theft detection, while R(2+1)D-18 offers the most convincing balance when compute is limited. The path forward is clear: consolidate the shared logic into tested modules, standardize evaluation and validate the complete service on real CCTV streams with transparent latency and drift monitoring.

References

[1] J. Carreira and A. Zisserman, “Quo Vadis, Action Recognition? A New Model and the Kinetics Dataset,” CVPR, 2017.

[2] D. Tran et al., “A Closer Look at Spatiotemporal Convolutions for Action Recognition,” CVPR, 2018.

[3] C. Feichtenhofer et al., “SlowFast Networks for Video Recognition,” ICCV, 2019.

[4] C. Feichtenhofer, “X3D: Expanding Architectures for Efficient Video Recognition,” CVPR, 2020.

[5] Z. Liu et al., “Video Swin Transformer,” CVPR, 2022.

[6] Y. Li et al., "MViTv2: Improved Multiscale Vision Transformers for Classification and Detection," CVPR, 2022.

[7] G. Bertasius, H. Wang and L. Torresani, "Is Space-Time Attention All You Need for Video Understanding?" ICML, 2021.

[8] M. Cheng, K. Cai and M. Li, "RWF-2000: An Open Large Scale Video Database for Violence Detection," ICPR, 2021.

[9] W. Sultani, C. Chen and M. Shah, "Real-World Anomaly Detection in Surveillance Videos," CVPR, 2018.

[10] D. D. de Paula et al., "CamNuvem: A Robbery Dataset for Video Anomaly Detection," Sensors, 2022.